

DATA ENGINEERING LAB (22MDCEL10)

CAT-1 PROJECT RECORD

**END-TO-END MOVIE DATA ENGINEERING
PIPELINE USING TMDB**



SUBMITTED TO

Dr. N. YAMUNA DEVI

SUBMITTED BY

Aditya (71762233001)

COIMBATORE INSTITUTE OF TECHNOLOGY

**(A Government Aided Autonomous Institution Affiliated to Anna
University) Coimbatore – 641 014, Tamil Nadu, India**

1. PROJECT TITLE

End-to-End Movie Data Engineering Pipeline using TMDB

2. INTRODUCTION

Data generated by modern applications and online platforms is continuously changing and needs to be collected, processed, validated, stored, and analyzed efficiently. A data engineering pipeline provides an automated mechanism to move data from its source to an analytical destination.

The objective of this project is to develop an end-to-end movie data engineering pipeline using real-time movie information obtained from **The Movie Database (TMDB) API**.

The project integrates the different stages of the data engineering workflow into a single pipeline. Movie data is extracted from TMDB using Python and is then passed through staging, validation, and transformation stages. The transformed records are published to an Apache Kafka topic and consumed by a Kafka consumer before being loaded into a SQLite database.

The pipeline also supports incremental data processing. Instead of inserting the same movie records repeatedly, the system checks the existing TMDB movie IDs in the SQLite database and processes only newly identified movies. This allows the database to grow incrementally.

Apache Airflow is used to automate and orchestrate the pipeline tasks, while Docker provides the execution environment for Airflow, Kafka, Zookeeper, and the Airflow PostgreSQL service.

Finally, Exploratory Data Analysis (EDA) is performed on the database, and the SQLite database is connected to Microsoft Power BI using an SQLite ODBC connection. The Power BI dashboard provides interactive visualizations of movie popularity, ratings, release years, languages, and other movie attributes.

The project therefore demonstrates a complete data flow:

TMDB API → ETL → Kafka → SQLite → EDA → Power BI

with Airflow responsible for workflow orchestration.

3. DOMAIN CHOSEN

Movie and Entertainment Analytics

The selected domain for the project is Movie and Entertainment Analytics.

The entertainment industry generates large volumes of data related to movies, including:

- Movie titles
- Release dates
- Genres
- Ratings
- Number of votes
- Popularity scores
- Original languages
- Movie descriptions
- Poster and backdrop information

Such information can be analyzed to understand audience preferences and movie trends.

The project uses TMDB as the external movie data source and focuses on analyzing:

- Movie popularity
- Movie ratings
- Number of votes
- Release-year trends
- Genre distribution
- Language distribution
- Most popular movies
- Relationships between numerical movie attributes

The domain was selected because it provides a practical use case for demonstrating API-based data extraction, ETL, incremental processing, streaming with Kafka, database storage, EDA, orchestration, and Business Intelligence visualization.

4. DATASET DESCRIPTION

4.1 Data Source

The primary data source is the “The Movie Database” (TMDB) REST API.

The project uses Python to communicate with the TMDB API and retrieve movie information dynamically.

The initial pipeline extracted **500** movie records.

Subsequent executions use an incremental extraction process to identify movies that are not already present in the SQLite database.

4.2 Dataset Attributes

The final movie dataset contains 14 columns:

Column	Description
tmdb_id	Unique TMDB movie identifier
title	Movie title
overview	Description/summary of the movie
release_date	Movie release date
popularity	TMDB popularity score
vote_average	Average rating of the movie
vote_count	Number of votes received
original_language	Original language of the movie
adult	Adult-content indicator
genre_ids	TMDB genre identifiers
poster_path	Movie poster path
backdrop_path	Movie backdrop path
created_at	Record creation timestamp
genres	Human-readable genre names

The genres column was added during the analytics process through genre-ID mapping.

4.3 Dataset Size

The initial dataset contained: **500 movies**

The incremental extraction mechanism was subsequently tested several times.

The database progressed as follows: **500 → 510 → 520 → 530 → 540 → 550**

The final database therefore contained: **550 movie records**

4.4 Data Quality

Data validation was implemented before the records were passed to the Kafka stage.

For one of the incremental batches:

- Records received: 10
- Valid records: 10
- Invalid records: 0

EDA also identified missing values. For example, the `backdrop_path` field contained a small number of missing values, which were reported as part of the data-quality analysis rather than being ignored.

5. TECHNOLOGIES USED

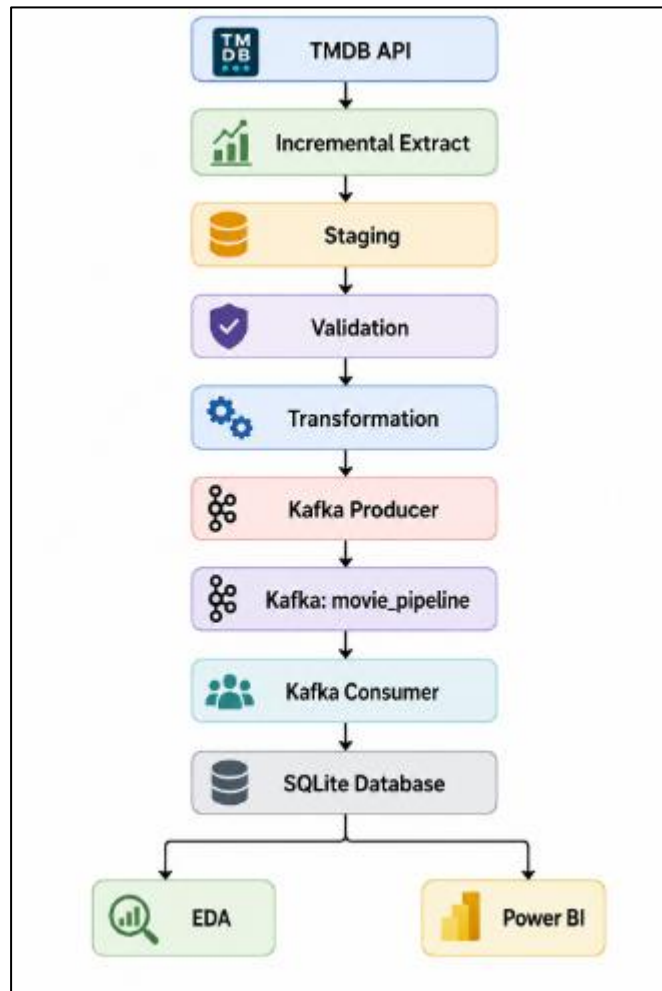
The following technologies were implemented in the project.

Technology	Purpose
Python	Data extraction, validation, transformation, Kafka processing, database loading and EDA
TMDB REST API	External movie data source
Pandas	Data processing and exploratory data analysis
Apache Kafka	Message streaming between transformation and database-loading stages
Zookeeper	Kafka coordination service used in the Docker deployment
SQLite	Persistent movie database
Apache Airflow	Pipeline orchestration and scheduling
Docker / Docker Compose	Deployment and execution environment
PostgreSQL	Airflow metadata database
Power BI	Business Intelligence dashboard and visualization
SQLite ODBC Driver	Connection between SQLite and Power BI
python-dotenv	Loading environment variables such as API and Kafka configuration
confluent-kafka	Python Kafka producer and consumer implementation

6. PIPELINE RELIABILITY AND DEPLOYMENT

6.1 Pipeline Architecture

The implemented pipeline follows the architecture:



Apache Airflow orchestrates these stages.

6.2 Initial and Incremental Extraction

The initial extraction collected 500 movie records.

The project then implemented `incremental_extract.py`.

The incremental process checks the existing movie IDs in SQLite and identifies movies that are not already present.

Only the newly identified records are passed through the remaining pipeline.

This prevents unnecessary processing of the complete dataset during every execution.

6.3 Staging

Separate staging and processing directories were created for different stages of the pipeline.

The project maintains:

```
data/  
├── raw/  
├── processed/  
├── staging/  
└── invalid/
```

This separation makes it possible to inspect intermediate pipeline outputs and identify problems at individual stages.

6.4 Validation

The validation stage checks incoming movie records before they continue to the transformation and Kafka stages.

Valid and invalid records are separated into different outputs.

This improves data quality and prevents invalid records from being directly inserted into the database.

6.5 Transformation

The transformation stage prepares validated movie records into a consistent structure required by the downstream Kafka and database components.

The transformation output is stored as processed JSON data.

6.6 Kafka Processing

Apache Kafka is used as the messaging layer.

The project uses the topic: **movie_pipeline**

The Kafka producer publishes transformed movie records to the topic.

The Kafka consumer listens to the topic and consumes the records before generating the staging output used by the database-loading stage.

During testing, Kafka successfully handled the incremental batches

For example:

- Publishing 10 movies to Kafka

- Movies delivered: 10
- Movies failed: 0

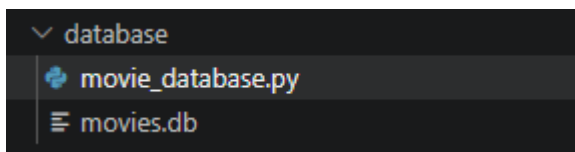
The consumer subsequently received all 10 records.

6.7 SQLite Database and Idempotency

SQLite is used as the target database.

The database-loading process checks whether a movie already exists before insertion.

This makes repeated executions safe because already existing movies are skipped.



For example, when a previously processed batch was sent to the database:

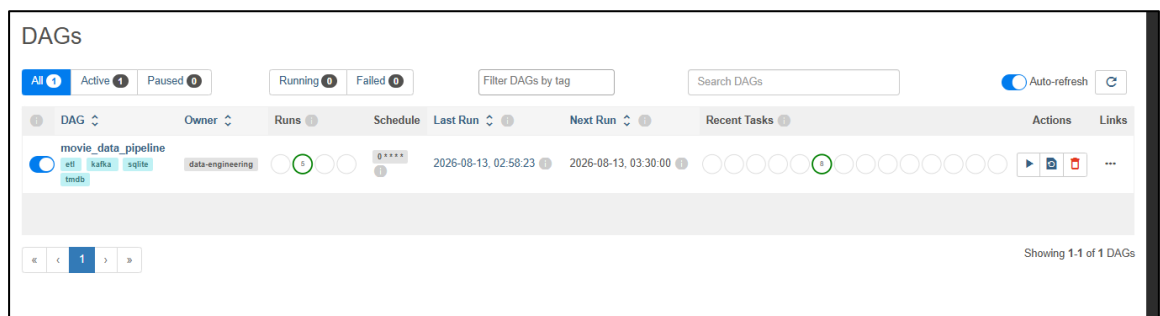
- Records received: 10
- Records inserted: 0
- Records skipped: 10
- Total movies in database: 520

This demonstrates duplicate prevention and incremental loading.

6.8 Airflow Orchestration

Apache Airflow is used to orchestrate the complete workflow.

The DAG is named: movie_data_pipeline

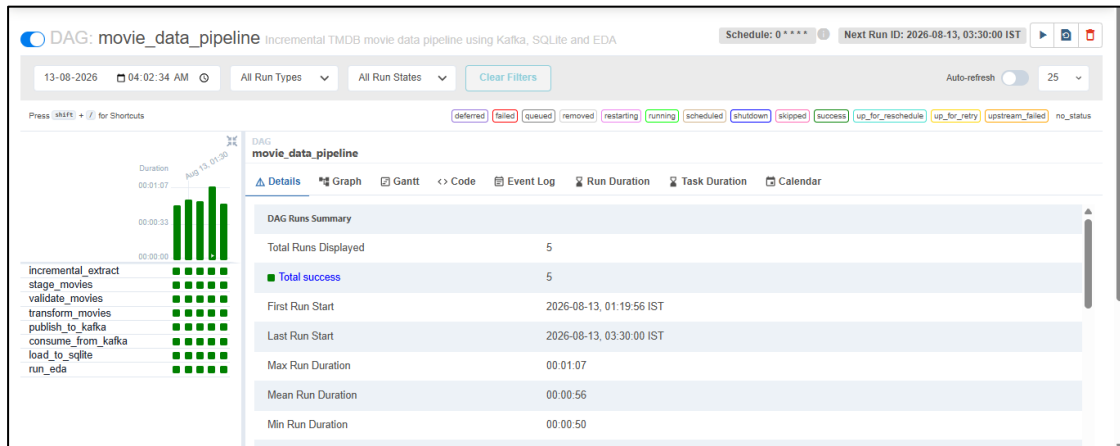


The implemented Airflow tasks are:

- incremental_extract
- stage_movies
- validate_movies
- transform_movies
- publish_to_kafka

- consume_from_kafka
- load_to_sqlite
- run_eda

The tasks execute in the required sequence.

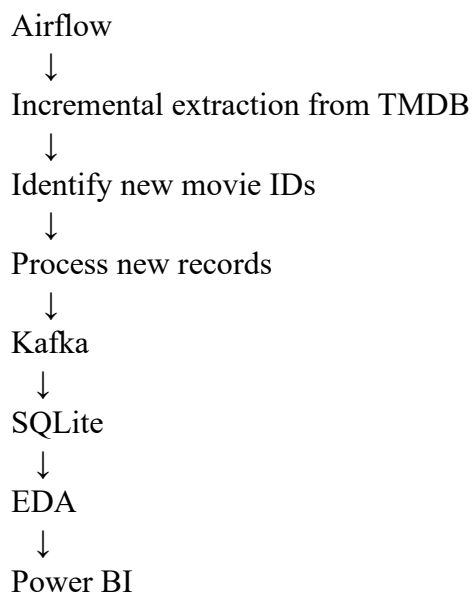


The complete DAG was successfully tested, with all eight tasks reaching the **SUCCESS** state.

6.9 Automatic Pipeline Execution

The pipeline is configured to run through Airflow scheduling. Therefore, new movie data does not need to be manually entered into the SQLite database.

The scheduled process performs:

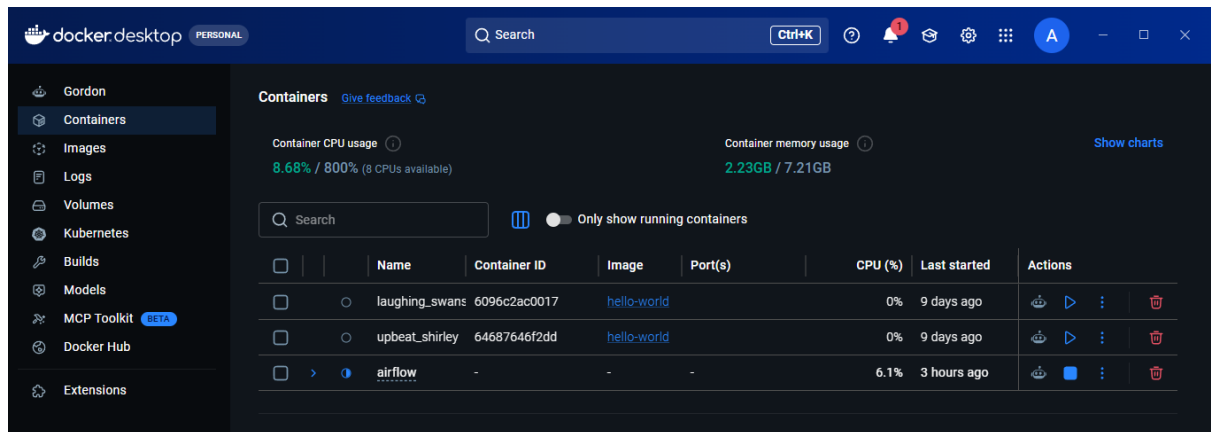


Manual DAG triggering was also used during testing to verify the complete workflow.

Docker Deployment

Docker Compose was used to deploy the infrastructure.

```
PS C:\Users\adity\Documents\Semester 9\Lab\Data Engineering Lab\Cat-1\Cat-1 Project\DE Project- Cat 1> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                                                                               NAMES
c81f1fac3458   airflow-airflow-scheduler           "/usr/bin/dumb-init ..." 3 hours ago   Up 3 hours   8080/tcp                                                  de-airflow-scheduler
6319e2b5202d   airflow-airflow-webserver           "/usr/bin/dumb-init ..." 3 hours ago   Up 3 hours   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp              de-airflow-webserver
367cd645d7b7   confluentinc/cp-kafka:7.5.3         "/etc/confluent/dock..." 3 hours ago   Up 3 hours   0.0.0.0:29092->29092/tcp, [::]:29092->29092/tcp          de-kafka
8f9e46bd9b23   postgres:15                         "docker-entrypoint.s..." 3 hours ago   Up 3 hours   5432/tcp                                                  de-airflow-postgres
2738bfa7c4a    confluentinc/cp-zookeeper:7.5.3     "/etc/confluent/dock..." 3 hours ago   Up 3 hours   2181/tcp, 2888/tcp, 3888/tcp                             de-zookeeper
```



The deployment contains:

- Airflow Webserver
- Airflow Scheduler
- Airflow PostgreSQL
- Kafka
- Zookeeper

This provides a reproducible environment for the pipeline's orchestration and streaming components.

7. DASHBOARD DEVELOPMENT AND ANALYTICS

7.1 Exploratory Data Analysis

EDA was performed using Python and Pandas after the movie data was loaded into SQLite.

The analysis includes:

- Dataset overview
- Data types
- Missing values
- Numerical summary
- Release-year analysis
- Rating statistics

- Vote-count statistics
- Popularity analysis
- Genre analysis
- Language analysis
- Correlation analysis
- Top popular movies

The EDA process also generates CSV reports in:
analytics/reports/

These include:

- numerical_summary.csv
- missing_values.csv
- release_year_analysis.csv
- genre_analysis.csv
- genre_summary.csv
- language_analysis.csv
- top_popular_movies.csv
- top_rated_movies.csv
- correlation_matrix.csv
- rating_correlation.csv
- overview.csv

7.2 Key EDA Findings

The final dataset contained 550 movies.

The analysis showed that:

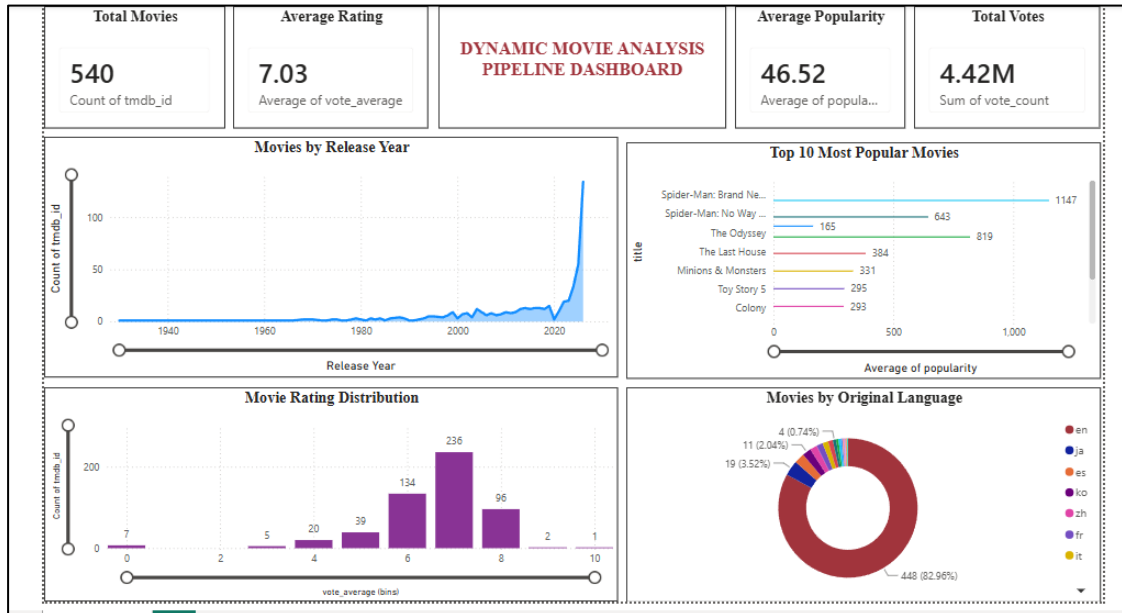
- English was the dominant original language.
- Action was the most frequently occurring genre.
- The dataset contained movies spanning several decades.
- Recent years, particularly 2024–2026, contained a significant proportion of the collected records.
- Movie popularity varied substantially between records.
- vote_average and vote_count showed a positive relationship.
- backdrop_path contained a small number of missing values.
- The adult field was constant at zero for the collected dataset.

7.3 Power BI Dashboard

The SQLite database was connected to Microsoft Power BI using an SQLite ODBC connection.

The dashboard contains KPI cards for:

- Total Movies
- Average Rating
- Average Popularity
- Total Votes



The dashboard also includes visualizations for:

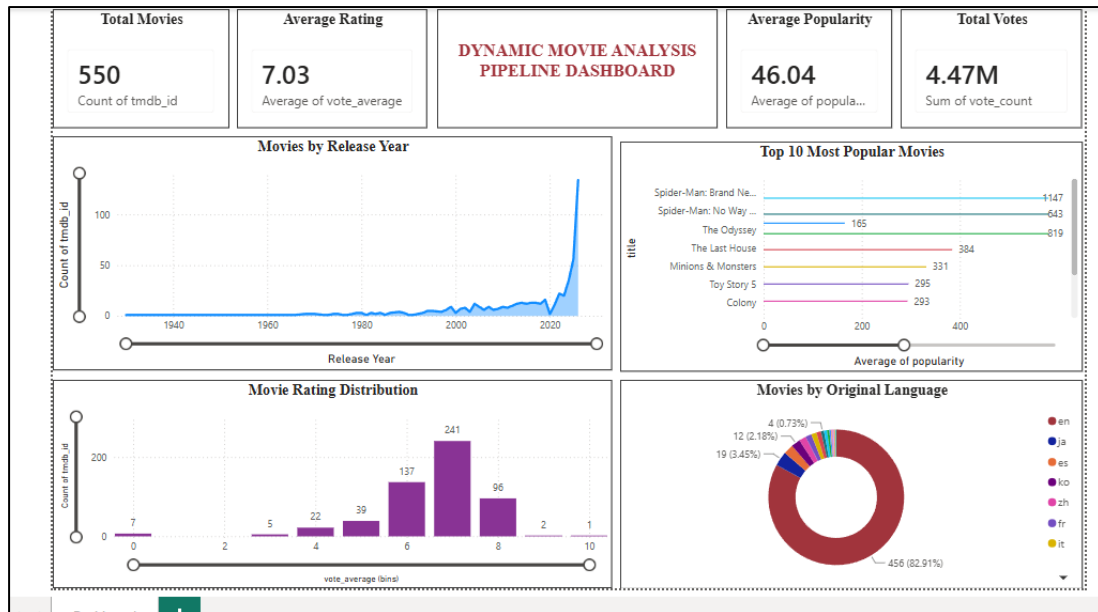
- Movies by Release Year
- Top 10 Most Popular Movies
- Rating Distribution
- Movies by Original Language
- Genre-related analysis

The dashboard provides an interactive analytical view of the movie dataset.

7.4 Dynamic Dashboard

The Power BI dashboard uses the SQLite database as its data source.

The dynamic behavior was demonstrated by processing additional movie records through the pipeline.



The database changed from: **540 movies** → **550 movies**

After refreshing the Power BI data, the dashboard reflected the updated dataset.

Therefore, the complete flow from incremental data ingestion to database update and BI visualization was demonstrated.

8. CONCLUSION

The **End-to-End Movie Data Engineering Pipeline using TMDB** successfully integrates the major components of a modern data engineering workflow into a single project.

The project begins with movie data extraction from the TMDB REST API and continues through staging, validation, transformation, Kafka-based message processing, SQLite database loading, exploratory data analysis, and Power BI visualization.

Apache Airflow provides workflow orchestration and scheduling, while Docker provides a consistent deployment environment for the pipeline infrastructure. The implementation also includes incremental extraction and duplicate prevention, allowing new movie records to be appended without repeatedly inserting existing records.

The pipeline was tested through multiple incremental executions, with the database growing from **500 movies to 550 movies**. The final Airflow execution successfully completed all eight pipeline tasks, including extraction, staging, validation, transformation, Kafka publishing, Kafka consumption, database loading, and EDA.

The project also demonstrates the connection between data engineering and Business Intelligence. The SQLite database acts as the persistent data layer for Power BI, and changes introduced through the incremental pipeline can be reflected in the dashboard after refreshing the data source.

Overall, the project successfully demonstrates an integrated **ETL and incremental data pipeline with Kafka, Airflow, Docker, SQLite, EDA, and Power BI**, satisfying the objective of building an end-to-end movie data engineering and analytics solution.